
IDATT2106, 2024 - Team 8

**Systemutvikling 2 med smidig prosjekt
Sluttrapport
Sparesti**

Versjon 3.0

Adele Kjølstad, Emil Slettbakk, Helene Nordby, Helle Rosenlind, Isak Sören Jonsson, John-Ivar Dalhaug
Eriksen, Knut Skoe og Magnus Gjerlaugsen

Thomas Selliseth og Thomas Lande

Revisjonshistorie

Dato	Versjon	Beskrivelse av endring	Forfatter
12/05/2024	1.0	Første utkast ferdig.	Adele Kjølstad, Emil Slettbakk, Helene Nordby, Helle Rosenlind, Isak Sören Jonsson, John-Ivar Dalhaug Eriksen, Knut Skoe og Magnus Gjerlaugsen
14/05/2024	2.0	Konklusjon, sammendrag og finpuss.	Adele Kjølstad, Emil Slettbakk, Helene Nordby, Helle Rosenlind, Isak Sören Jonsson, John-Ivar Dalhaug Eriksen, Knut Skoe og Magnus Gjerlaugsen
15/05/2024	3.0	Ferdigstilling.	Adele Kjølstad, Emil Slettbakk, Helene Nordby, Helle Rosenlind, Isak Sören Jonsson, John-Ivar Dalhaug Eriksen, Knut Skoe og Magnus Gjerlaugsen

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Sammendrag

Denne rapporten gir en detaljert gjennomgang av utviklingsprosessen for Sparesti, en app som kombinerer spillifisering med økonomisk opplæring for å fremme sparing. Appen benytter flere grunnprinsipper innenfor programmering og er laget for å tilpasse seg individuelle brukere og deres økonomiske mål. Universell utforming og WCAG-regler er også tatt med i utviklingen for å sikre at innholdet er tilgjengelig uavhengig av funksjonsevne. Rapporten dekker prosessen og diskuterer teamets beslutninger, erfaringer og refleksjoner rundt det endelige produktet.

Forord

Rapporten er skrevet for Norges teknisk-naturvitenskapelige universitet i emnet IDATT2106 Systemutvikling med smidig prosjekt på vegne av instituttet for datateknologi og informatikk. Målet for prosjektet er å lage en applikasjon for å motivere individer mellom 18-60 år til å spare penger, samt å bidra til å øke deres kunnskap rundt personlig økonomi. Arbeidet er gitt av produkteiere fra masterprogrammet Digital transformasjon og har som hensikt å lage et brukervennlig program som kan være med på å skape bedre bevissthet rundt økonomi og sparing for brukerne sine.

Underveis i utviklingen har teamet hatt god kommunikasjon med produkteiere og jobbet agilt for å tilrettelegge deres krav og visjoner. Gjennom daglige stand-up møter har teamet også skapt god kommunikasjon og samarbeidsvillighet, som har kommet godt med i prosjektet. Medlemmene har tilegnet seg ny kunnskap om teamsamarbeid og scrum-arbeidsmetodikk, samtidig som de har fått trening i et mer profesjonelt klientoppdrag, som vil gi verdifull erfaring for videre arbeid. Teamet vil rette en stor takk til produkteierne Thomas Selliseth og Thomas Lande for tilbakemeldingene og for å legge frem visjonen for produktet. En stor takk rettes også til Pedro P.C. Arroyave for hans veiledning underveis i arbeidet.

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Signaturer

Adele Kjølstad

Adele Kjølstad

Emil Slettbakk

Emil Slettbakk

Helene Nordby

Helene Nordby

Helle V. Rosenlind

Helle Rosenlind

Isak Jonsson

Isak Sören Jonsson

John I. Eriksen

John-Ivar Dalhaug Eriksen

Knut Skoe

Knut Skoe

Magnus Gjerlaugsen

Magnus Gjerlaugsen

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Innholdsfortegnelse

Sammendrag	3
Forord	3
Oppgavebeskrivelse	6
1. Introduksjon	6
1.1 Akronymer og forkortelser	7
2. Teori og relevant litteratur	7
3. Metode	9
4. Resultater	13
4.1 Prosjektresultater	13
4.2 Administrative resultater	16
5. Konklusjon og videre arbeid	16
5.1 Diskusjon	17
5.2 Videre arbeid	19
5.3 Konklusjon	19
6. Referanser	19

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Oppgavebeskrivelse

Oppgaven går ut på å utvikle “Sparesti”, en webapplikasjon med mål om å motivere voksne til å forbedre egen økonomi og øke bevisstheten over eget forbruk gjennom et budsjett. (Visjonsdokument-Sparesti, 2024). Prosjektet foregår i en tre-ukers periode, der produkteierne er masterstudentene fra digital transformasjon, og står for visjon og mål for applikasjonen.

Utfordringer med å spare kommer gjerne fra mangel på kunnskap og oversikt over egen økonomi, der tradisjonelle sparemetoder eller råd ikke tar hensyn til den enkeltes finansielle situasjon og sparemål. Uten kunnskap og forståelse av hvordan penger skal styres på en fornuftig måte kan det påvirke økonomien negativt. Applikasjonen skal være rettet mot mennesker mellom 18-60 år, der en vellykket løsning skal bedre deres motivasjon til sparing og gjøre at de tilegner seg mer kunnskap rundt eget forbruk. Målet er at appen skal bidra til mer sparing, gjøre brukere mer bevisst på egen økonomi og forbruk, samt være kunnskapsgivende, der det skal være tilgjengelige tips og nytt fra det økonomiske nyhetsbildet. (Sparesti, 2024)

Det som er unikt for Sparesti er at den skal generere persontilpassede spareutfordringer gjennom data om forbruket. I tillegg skal man kunne sette opp egne mål og være funksjonalitet for budsjettering. Gjennom dette er formålet å spillifisere sparemetoder, så det skal være motiverende og gøy for brukerne å spare opp til målene sine. Applikasjonen skal være tilpasset standard PC-skjerm, men skal også fungere på standard mobilskjerm og plattformen må fungere i de vanligste nettleserne. (Sparesti, 2024). Videre er ikke-funksjonelle tekniske krav definert i visjonsdokumentet og samleforelesningen (IDATT2106 - Prosjekt, 2024).

1. Introduksjon

Denne rapporten er utarbeidet som en del av faget IDATT2106 - Systemutvikling 2 med smidig prosjekt ved Norges teknisk-naturvitenskapelige universitet (NTNU) for 2024. Oppgaven ble tildelt 15. april og ferdigstilt 16. mai og hadde som mål å adressere vanlige utfordringer folk møter med sparing. Dette skulle gjøres med en applikasjon som kan tilby persontilpassede sparetips og utfordringer til hver enkelt bruker, samtidig som den skal fremstå som en motiverende og morsom personlig økonomistyrings-app. Arbeidet er utviklet i samarbeid med masterstudenter i Digital Transformasjon som sto for visjonen og målet for applikasjonen og hadde rolle som produkteiere.

Produktet, Sparesti, skal gjøre sparing morsomt ved å gi personlige utfordringer, motivasjon og tips basert på brukerens ønsker. Dette har som formål å løse problemer med sparing, da det ofte innebærer mangel på personalisering, motivasjon og mangel på kunnskap/kompetanse. Applikasjonen har et enkelt brukergrensesnitt og gir brukere mulighet til å lage sparemål, motta persontilpassede utfordringer, få en oversikt over penger spart, vise fremgangen visuelt og få badges for motivasjon. Et mer detaljert sammendrag over krav finnes i visjonsdokumentet for Sparesti (Sparesti, 2024). Produkteierne spilte også en sentral rolle i utvikling og definerte hvilke funksjonaliteter og krav de ønsket at teamet skulle prioritere.

Prosjektet er utformet med et visuelt grensesnitt der applikasjonen kan kjøres lokalt eller tilgjengeliggjøres gjennom domenet sparesti.net. Utviklingsprosessen ble gjennomført i to sprinter og var i samsvar med kravet om arbeidsmetodikk. Prosessen fulgte en iterativ prosess for evaluering og tilpasning for å sikre at produktet var i samsvar med produkteiernes behov og tilbakemeldinger. Denne rapporten inneholder en analyse og beskrivelse av prosjektførsløpet. Dette inkluderer teori, selve prosessen, endringer og utviklingen av applikasjonens arkitektur og funksjonalitet, diskusjon og tanker rundt videre utvikling og til slutt konklusjon.

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

1.1 Akronymer og forkortelser

- SPA - Single Page Application, er en metode for å implementere en webapplikasjon. Metoden går ut på at nettleser bare henter en nettside fra tjener, som deretter blir oppdatert av nettleser dynamisk med ressurser fra tjener istedenfor at helt nye nettsider blir hentet fra tjener. (Fink et al., 2014, s. 11).
- JPA - Java Persistence API. En spesifikasjon for Java som håndterer databasetilgang, persistens og datastyring mellom Java-objekter og relasjonelle databaser.
- REST - Representational State Transfer. En arkitektonisk stil for å utforme nettverksapplikasjoner ved hjelp av såkalt "stateless" forespørsler og overføringer av representasjoner av ressurser.
- IDE - Integrated Development Environment. Et integrert utviklingsmiljø som kombinerer flere viktige og vanlige utviklerværktøy i et og samme program.
- GDPR - General Data Protection Regulation. En regulering i EU-lovgivning for databeskyttelse og personvern for individer innenfor EU og EØS.
- API - Application Programming Interface. Et sett med regler og definisjoner som legger til rette for kommunikasjon mellom forskjellige typer programvare.
- CSS - Cascading Style Sheet. Et språk som beskriver stil og presentasjonen av dokumenter skrevet i HTML eller XML.
- HTML - HyperText Markup Language. Standardspråket som brukes til å lage grunnleggende nettsider.
- MySQL. Et relasjonelt databasesystem.
- LTS - Long Term Support. En betegnelse for programvareversjoner som tilbyr utvidet støtte og vedlikehold over en lengre periode. Ofte målrettet mot bedrifter og organisasjoner som krever stabile plattformer.
- PO - Product Owner. Produkteier på prosjekter. Den eller de som har hovedansvaret for prosjektet på oppdragsgiverens side.

2. Teori og relevant litteratur

Smidig programvareutvikling

Som bakgrunn for metodikken som ble benyttet i prosjektet ligger teoriene for smidig programvareutvikling. Begrepet smidig programvareutvikling omfatter flere ulike metoder som for eksempel Scrum. Manifestet for smidig programvareutvikling er hovedmålet å "finne bedre måter å utvikle programvare på ved å gjøre det selv og ved å hjelpe andre med det." (Agile Alliance, 2001). Resten av manifestet beskriver hvordan dette gjøres: "Personer og samspill fremfor prosesser og verktøy. Programvare som virker fremfor omfattende dokumentasjon. Samarbeid med kunden fremfor kontraktsforhandlinger. Å reagere på endringer fremfor å følge en plan." Og som det også presiseres i slutten av manifestet har punktene som her blir nedprioritert verdi, men de andre punktene verdsettes enda høyere. Manifestet ble utarbeidet av representanter fra blant annet Scrum, DSDM, Extreme Programming og andre som ønsket å presentere et alternativ til dokumentasjonsdrevet programvareutvikling.

Wireframe

Som en del av utviklingsprosessen er det hensiktsmessig å vite hva en skal lage og hvordan det skal se ut så tidlig som mulig i arbeidsprosessen. Løsningen på dette kan være en Wireframe. En Wireframe er en enkel representasjon av det ferdige produktet og trenger ofte ikke inneholde mange detaljer. Det viktige er at den ikke er energi- og tidkrevende å produsere. Noen vanlige ting å ta med i en Wireframe er overordnet oppsett av applikasjonen, navigering mellom ulike sider og Interaktive elementer. Wireframen brukes gjerne til å få tidlige tilbakemeldinger om brukergrensesnittet og generell brukeropplevelse. Etter hvert som en får tilbakemeldinger kan en lage nye Wireframer som inneholder mer detaljer, før en til slutt ender opp med noe som ser ut som det ferdige produktet. (What is wireframing?, u.å)

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Brukertesting

I et slikt prosjekt er brukertesting også et viktig verktøy å benytte i utviklingsprosessen. Brukertesting kan brukes til å få kunnskap om hvordan produktet fungerer, diskusjons- og konklusjonsgrunnlag for design-avgjørelser og enighet om hvilke behov produktet skal dekke (Givær, u.å.). Brukertesting er viktig i alle faser av et prosjekt for å se at produktet utvikles i riktig retning. Det er derfor viktig at det utføres brukertesting fra tidlig i prosessen slik at man unngår å måtte gjøre store endringer senere i prosjektet. Lenger ut i prosjektet vil kostnaden ved store endringer også øke og er dermed enda et argument for å drive kontinuerlig brukertesting (Grolid, 2023). Etter produktlansering er det også viktig å fortsette med brukertesting for å kunne videreutvikle produktet til å møte brukernes behov.

Universell utforming

Et annet viktig hensyn å ta i utvikling av applikasjoner i dag handler om universell utforming. Det vil si at applikasjonen skal gjøres tilgjengelig å bruke for så mange mennesker som mulig med de hensynene som må tas. I universell utforming ønsker man at tjenester skal være tilgjengelig for alle uavhengig av alder, funksjonsevne og utdanningsnivå. I et stadig mer digitalisert samfunn er det dermed også viktig at de digitale løsningene tilpasses slik at de kan brukes av flest mulig. I Norge er Digitaliseringsdirektoratet pålagt å følge opp at blant annet nettsider og apper følger regelverket for universell utforming av IKT (Tilsynet for universell utforming av ikt, u.å. a). Dette regelverket er bygget på WCAG-standarden som definerer minimumskrav for en universelt utformet digital plattform (Tilsynet for universell utforming av ikt, u.å. b). Eksempler på krav her kan være å tilby teksting av forhåndsinnspilt video, og at man ikke skal benytte presentasjon som utelukkende bygger på farger (Tilsynet for universell utforming av ikt, u.å. c).

GDPR

General Data Protection Regulation (GDPR) er en streng lov som regulerer personvern og databeskyttelse for enkeltpersoner innenfor EU og EØS. GDPR trådte i kraft i 2018 og erstattet den tidligere databeskyttelsesloven fra 1995. Målet med GDPR er å gi enkeltpersoner større kontroll over hvordan deres personlige data blir samlet inn, behandlet, lagret og brukt av organisasjoner. Loven pålegger strenge straffer ikke bare innenfor sine egne land, men til alle organisasjoner i verden som behandler data rettet mot innbyggere i EU og EØS. Noen av hovedprinsippene i GDPR inkluderer:

- At behandling av data må være lovlig, rettferdig og transparent.
- Innsamlet data skal kun brukes til det som brukeren har fått beskjed om.
- Det skal kun samles inn så mye data som er nødvendig.
- Innsamlet data må være nøyaktig.
- Innsamlet data skal kun lagres så lenge det er nødvendig.
- Lagring av data skal gjøres så det sikrer hensiktsmessig konfidensialitet, sikkerhet og integritet.
- De som er ansvarlig for lagring av data må kunne vise at de tilfredsstiller kravene i GDPR.

(Wolford, u.å.)

Relasjonsdatabaser

En relasjonsdatabase er en database som knytter sammen informasjon lagret i tabeller med relasjoner basert på felles data eller en samling med normaliserte relasjoner som har distinkte navn (Hansen & Mallaug, 2003, s. 48). At databasen er på normalform vil kortfattet si at databasen er delt opp i mindre tabeller basert på gitte regler. Disse reglene sørger for at hver rad/kolonne inneholder kun en atomisk verdi. Kandidatnøkkelen determinerer alle ikke-nøkkelkolonner og at en ikke-kandidatnøkkelkolonne ikke kan determineres av noen annen kolonne utenom

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

primærnøkkel. Det er noen unntak, men i de fleste tilfeller vil disse kriteriene sørge for at databasen er normalisert (Hansen & Mallaug, 2003, s. 86-93).

Relasjonsdatabaser er egnet til strukturert data, gir god fleksibilitet og sørger for dataintegritet. Dersom datasettene er store eller ustrukturerte, kan modellen begynne å få problemer.

3. Metode

Metodikk

Scrum ble brukt som grunnlag for utviklingsprosessen til prosjektet. Metoden er beskrevet i rapportens teoridel. Teamet praktiserte metoden ved å velge en scrum master og ved å praktisere flere av seremoniene som metoden innebærer. Før utvikling hadde teamet planlegging av produktkø sammen med produkteiere. I løpet av denne delen av prosessen definerte teamet brukerhistorier og prioriteringer av disse sammen med produkteiere som ble grunnlag for videre planlegging av hver sprint i prosjektperioden. Under sprint planleggingen satte teamet opp en sprintkø med mindre oppgaver basert på de brukerhistoriene som teamet valgte å jobbe med for hver sprint. Disse oppgavene fikk også hver en estimert mengde med innsats under planleggingen som teamet bestemte i fellesskap. Teamet tok i bruk estimat i form av timer og i form av tall som representerte kompleksiteten til hver oppgave i forhold til andre. Videre ble det også holdt stand-up-møter hver dag der det var satt av tid til å diskutere fremskritt og det som har blitt gjort. Teamet produserte også et Burndown Chart i løpet av den andre sprinten. Etter hver sprint ble det også utført et sprint review og retrospektiv. Sprint review ble gjort sammen med produkteier og innebar å vise frem produktets status og få tilbakemelding for videre arbeid. Teamet brukte sprint retrospektiv til å se på forbedringspunkter og hva teamet kunne ta videre av utviklingsprosessen og teamsamarbeidet. For å holde orden på status til oppgaver under hver sprint brukte teamet et Issue Board. Dette ble oppdatert fortløpende av hele teamet. Utover bruken av Scrum-metoden, strukturerte teamet seg også i to mindre team, som hvert hadde ansvar for utvikling av en del av applikasjonen. For å holde kommunikasjonen gående hadde teamet statusmøter etter lunsj og felles overblikk av Issue Board på slutten av dagen hver dag under sprint 2.

Arbeids- og rollefordeling

I henhold til Scrum-metoden ble det valgt en Scrum Master, som var ansvarlig for å lede de forskjellige Scrum-seremoniene som Daily Standup, planlegging av Sprint, Sprint retrospektiv og møter. I tillegg fungerte Scrum Master som "Team Lead", det vil si bindeledd mellom teamet og produkteierne. I forkant av prosjektstart ble det gjennomført møte hvor ansvarsområder ble fordelt, hovedsakelig dokumentansvarlig, referent og Scrum Master. Ut over dette var det ikke noen eksplisitte roller som hadde praktisk betydning for arbeidet, og teamet gikk inn for en relativt flat, tillitsbasert struktur.

I planleggingen til Sprint 1 ble det satt opp brukerhistorier i fellesskap. Deretter delte teamet seg i to grupper, hvor den ene utformet en Wireframe basert på brukerhistoriene, og den andre delte brukerhistoriene opp i mindre oppgaver og satte opp Backlog basert på disse. Deretter ble de to delene gjennomgått i fellesskap.

Da utviklingsarbeidet startet, delte teamet seg i to mindre team, ett for Backend og ett for Frontend. Innad i disse var det heller ingen eksplisitte roller og ansvarsområder, medlemmene jobbet ut fra prinsippet om at hver person velger oppgaver fra køen de føler de mestrer eller er interessert i å lære om. En vesentlig del av å få dette til å fungere var at alle var trygge på å si ifra om de trengte hjelp eller ikke fikk til noe, slik at ikke oppgaver tok unødvendig lang tid å ferdigstille, hvilket kunne gjort at hele prosjektet ble forsinket. En slik åpen rollefordeling ble valgt med bakgrunn i at alle da hadde anledning til å prøve seg på forskjellige deler av utviklingen, uten å på forhånd være bundet til en spesifikk funksjon, med mål om at det da ble tryggere å prøve seg på ting litt utenfor komfortsonen.

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Selv om alle i utgangspunktet sto fritt til å velge sine egne oppgaver, var det en felles enighet om, og villighet til, å bli tildelt oppgaver, enten fordi dette var noe vedkommende var god på eller at det bare var noe som økte i prioritet og var nødvendig for prosjektets progresjon.

Prosjektkrav

Oppgaven ble gitt med krav til bruk av både spesifikke teknologier og metoder. For eksempel bruk av Java og Spring Boot for backend, RESTful-API, og Scrum som arbeidsmetode. På frontend omhandlet kravene i stor grad brukerinteraksjon, for eksempel at applikasjonen skulle oppfylle kravene til WCAG 2.2, altså legges til rette for universelt design, krav om plattformuavhengighet med hensyn på nettleser og krav til sikkerhet for passord og brukerdata. Krav til frontend innebar også at det skulle utvikles som en webbasert SPA. Slike tekniske krav var tydelig formulert i visjonsdokumentet.

Ut over de tekniske kravene, lå det mange mer eller mindre eksplisitte funksjonalitetskrav innbakt i visjonsdokumentet fra produkteier. For å identifisere disse ble det utformet brukerhistorier basert på visjonsdokumentet. Med utgangspunkt i brukerhistoriene ble det satt opp en Wireframe-modell av applikasjonen, i tillegg til en Backlog hvor hver brukerhistorie ble brutt opp i mindre deloppgaver og funksjoner. Disse ble presentert for produkteier, sammen med brukerhistoriene, hvilket gjorde at de kunne komme med konkrete tilbakemeldinger, og eventuelt gjøre endringer om noe ikke var i tråd med produkteiers ønsker. Bruk av Wireframe tidlig i planleggingen lot produkteier få et konkret inntrykk av teamets tolkning av oppgaven, hvilket bidro til at de lettere kunne identifisere hva som stemte og hva som manglet i teamets tolkning av visjonen. Gjennom dialog og møter med produkteier ble kravene og prioriteringene endret underveis, etter hvert som systemet tok form og produkteier fikk bedre inntrykk av hva resultatet kunne bli.

System og arkitektur

Systemets arkitektur baserte seg på en backend utviklet med Java, Spring Boot og Java Persistence API (JPA). Java tilbyr en robust plattform for å utvikle server-side-tjenester, mens Spring Boot gjør det enklere å sette opp og konfigurere applikasjonen ved å tilby en rekke forhåndskonfigurerte innstillinger som automatisk setter opp applikasjonen i henhold til gode og etablerte standarder. Her spiller annotasjoner en nøkkelrolle, ved å redusere behovet for å skrive eksplisitt konfigurasjon og såkalt “boilerplate”-kode. Dette ble brukt sammen med lagdeling, med blant annet Service-, Model- og Repository-pakker, for å separere ansvarsområder.

Brukerdata ble lagret i en MySQL database for å sikre smidig og sikker lagring av data. Sensitiv informasjon blir lagret kryptert og CASCADE er satt opp slik at all informasjon knyttet til en bruker kan bli fjernet. Disse grepene er gjort for å bedre sikkerheten og ytelsen til databasen, men også for å tilfredsstille krav om håndtering av persondata i henhold til GDPR.

For kommunikasjon med frontend, eksponerte backend RESTful API-enderpunkter. Disse ble brukt for å manipulere data, via HTTP-metoder som GET, POST, PUT og DELETE. RESTful gir gode retningslinjer for hvordan disse skal brukes for å oppnå konsistent og effektiv interaksjon mellom klient og tjener, noe som sikrer god forutsigbarhet og struktur i programmets data og operasjoner.

Frontend ble utviklet med Vue3 Composition API, HTML, CSS og Javascript. Her danner Vue3 et rammeverk som gjør det enklere å bygge interaktive brukergrensesnitt, ved å gi utviklere et enklere og mer oversiktlig oppsett for både HTML, CSS og Javascript. Vue3 bygger også på konseptet med komponentbasert design. Det tillater at elementer som håndterer spesifikke deler av grensesnittet kan gjenbrukes flere steder. Grensesnittet ble lagt opp til responsivt design, slik at det skulle fungere like godt på både PC og mobil, men med hovedfokus på mobile enheter, i henhold til produkteiers ønsker. For å sikre god tilgjengelighet ble fargevalg sjekket opp mot kjente fargepaletter som oppfyller krav fremsatt i WCAG med hensyn på fargeblindhet og svaksynte. Disse retningslinjene har også den

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

fordelen at de legger til rette for generelt gode designvalg, men hensyn på farger, skrifttyper og -størrelse, kontrast osv.

Brukertester

Underveis ble det gjennomført brukertester på applikasjonen. Disse ble i hovedsak gjennomført ved at teamets medlemmer rekrutterte bekjente som var villige til å gjennomføre brukertesten. For å få gode tester ble det benyttet et felles skjema, med noen konkrete spørsmål og brukerens inntrykk av programmets design, både det estetiske og hvorvidt det opplevdes som funksjonelt. I tillegg ble testpersonen gitt noen oppgaver de skulle utføre, uten veiledning. Dette er en veldig sentral del av gode brukertester, nemlig å observere hvordan en person, naiv til dette spesifikke miljøet, navigerer og opptrer. Videre ble MVP, ved slutten av sprint 1, testet på produkteier og fagveileder. Produkteiers test bidro til å informere produktets retning og videre prioriteringer, mens fagveileders test gav nyttige tilbakemeldinger av mer teknisk art.

Litteratur og ressurser

På det mer teoretiske planet, ble kunnskap og leksjoner fra årets pensum brukt, i tillegg til kunnskap opparbeidet fra tidligere semester. Spesielt var introduksjonen og arbeidet med universelt design og WCAG, fra IDATT1002, nyttig å se tilbake på. Kombinert med nettsteder som Uutilsynet, gir dette god innsikt i viktige prinsipper, for eksempel hvilke farger og skrifttyper som er gode valg, både for generelt godt design og for å være i tråd med gjeldende krav for tilgjengelighet og universelt design. Teamet har således hentet inspirasjon fra tidligere prosjektoppgaver, spesielt fra IDATT1002 Systemutvikling 1, hvor det ble utviklet et program, med tilhørende Gitlab, rapport og wiki.

Programmeringen av frontend og backend var direkte relatert til pensum i IDATT2104, IDATT2105 og IDATT2106. Der pensumlitteraturen ikke strakk til, var nettsteder som W3Schools, Baeldung og GeeksForGeeks gode kilder til eksempler og lærings artikler på hvordan funksjoner kunne settes opp, og ikke minst StackOverflow, som inneholder spørsmål og svar fra brukere på en mengde programmering relaterte områder. På frontend var spesielt W3Schools og Youtube-kanaler som NetNinja flittig brukt til å undersøke funksjonalitet og finne løsninger.

Kunstig intelligens, som Github Copilot og ChatGPT, må nevnes, og disse ble benyttet sammen med de andre nevnte nettstedene. I hovedsak ble de brukt til å stille spørsmål om mulige løsninger på problemer, eller til å hjelpe å identifisere feil i koden. Å bruke LLM til programmering er enn så lenge noe begrenset, og er ingen erstatning for faktisk forståelse, men det er en verdifull ressurs å bruke om man trenger forslag til hvordan et problem kan løses, og så tilpasse de mulighetene som presenteres. Teamet hadde ikke satt noen begrensninger eller retningslinjer for bruk av LLM, og lot det være opp til den enkelte å utvise skjønn i bruk av det. Som nevnt er ikke disse verktøyene på et nivå hvor de produserer kode som er fullgod i seg selv, og de krever menneskelig innsikt og forståelse for hvordan kodeforslagene skal integreres i resten av prosjektet. Dermed ble det ikke ansett for å utgjøre en vesentlig risiko for prosjektets integritet, den tid hverken ChatGPT eller Copilot er i stand til å produsere leveringsklar kode for et såpass komplekst system.

Programvare og verktøy

De integrerte utviklingsmiljøene (IDE-ene) IntelliJ og WebStorm fra JetBrains ble brukt under prosjektutviklingen. Versjonskontroll ble håndtert gjennom universitetets GitLab-domene, mens MySQL og PhpMyAdmin ble brukt for å administrere og få tilgang til databasen. Valg av programmeringsspråk ble basert på den nyeste tilgjengelige "Long-Term Support" (LTS)-versjon. I verktøyene som ikke opererer med LTS-utgivelser, ble nyeste stabile versjon valgt. Dette ble gjort for å unngå oppdateringer som kunne skape problemer med eksisterende kode, og for å dra nytte av forbedringer relatert til ytelse og sikkerhet. På dette grunnlaget ble Java 21 valgt for backend, Vue 3 for frontend, og den nyeste stabile versjonen av alle Spring Boot-avhengigheter. Videre ble byggeverktøyene Maven og

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Vite brukt for å bygge henholdsvis backend og frontend. Til testing av kode falt valget på rammeverkene JUnit5 for backen og Cypress for frontend.

For å lage Wireframe ble Figma brukt, og lagring av dokumenter ble gjort på Google Drive. Kommunikasjon foregikk via en egen meldingsgruppe på Facebook Messenger. Tekstdokumenter og regneark ble gjort på Google Docs og Spreadsheet. Øvrig e-post og møteinnkalling ble sendt via student-epost på NTNU sin Outlook-tjeneste.

Dokumentasjon

For dokumentasjon ble JavaDoc benyttet på backend, mens frontend brukte JSDoc. Dokumentasjonen av koden ble utført underveis i arbeidet for å kommunisere mangler eller forbedringer med andre teammedlemmer, og en siste gang etter at klassen var ferdigstilt for å fjerne unødvendige kommentarer og gi en bedre beskrivelse av funksjonen til forskjellige klasser og metoder. Hensikten med dette var å gjøre koden mer oversiktlig slik at videre utvikling eller vedlikehold av prosjektet vil bli enklere. RESTful endepunkter eksportert av backend ble også dokumentert med programvaren Swagger (SmartBear Software Inc., 2024).

I tillegg fungerer prosjektets wiki-side som dokumentasjon av systemet, utviklingen og funksjonalitet, med blant annet relevante UML-diagrammer, brukertester og brukerinstruksjoner.

4. Resultater

4.1 Prosjekresultater

I sprint 1 ble seks av de ni oppsatte brukerhistoriene påbegynt. Se vedlegg 4 for brukerhistoriene og vedlegg 5 og 6 for sprintkøene. Selv om ingen av brukerhistoriene var fullstendig ferdige etter sprint 1, var flere av akseptansekriteriene fullført og resten ble videreført til sprint 2.

Som bruker ønsker jeg et intuitivt og oversiktlig brukergrensesnitt for å navigere til de forskjellige funksjonene.

Tidlig i Sprint 1 ble wireframen fullført, se vedlegg 7, og deretter ble frontend satt opp. Alle sidene som var planlagt fra wireframen ble satt opp med navigasjonsmuligheter mellom dem. I tillegg ble fargevalgene testet i henhold til WCAG retningslinjene. Tilbakemeldingene fra brukertestene av wireframen, se vedlegg 8, 9 og 10, pekte på behovet for tilbakeknapper på enkelte sider, samtidig som det ble påpekt at hamburgermenyen ikke var nødvendig. Disse tilbakemeldingene ble implementert i løpet av Sprint 1. Videre ble det positivt bemerket at tekststørrelsen var passende, det var positivt at farger fra logoen ble gjentatt i resten av nettsiden, og ikonene ble ansett som intuitive. I første sprint var utformingen for mobil i fokus. Responsivt design for både PC og mobil ble jobbet videre med i sprint 2.

Som bruker ønsker jeg å opprette en brukerkonto slik at jeg kan logge inn og få oversikt over mine sparemål og økonomi.

Mulighet for å opprette en bruker, velge kunnskapsnivå, logge inn med brukernavn og passord og få en e-post med nytt passord hvis man har glemt det ble ferdigstilt i den første sprinten. Konfigureringen av brukerkontoen ved registrering var delvis ferdig etter første sprint. Valg av kunnskapsnivå og opprettelse av mål og utfordringer var på plass, og så ble valg av villighetsnivå til å endre vaner og mulighet for å laste opp csv-filer med kontoutskrifter implementert i sprint 2. Innlogging med BankID og OAuth-kontoer som Google osv. ble satt opp i backloggen til sprint 2, men etter samtale med produkteier ble disse prioritert vekk og ikke gjennomført. For å oppsummere er det bare akseptansekriteriet om å kunne velge mellom flere innloggingsmetoder som ikke ble fullført i sluttproduktet.

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Som bruker av programmet ønsker jeg å opprette egne sparemål slik at jeg kan få motivasjon til å spare.

I sprint 1 ble mulighet for å opprette et sparemål med navn, sum og sluttdato implementert i backend. Mål-siden i frontend viste en oversikt over alle målene til brukeren, inkludert hvor mye penger som er spart opp til hvert mål, og man kunne trykke på et mål for å få opp de tilhørende utfordringene. I sprint 2 ble denne løsningen byttet ut med Sparesti-siden.

Som bruker ønsker jeg å se en oversikt over penger jeg har spart slik at jeg ser fremgangen min.

Informasjon om hvert enkelt sparemål ble vist på Mål-siden i sprint 1. Visning av den totale summen brukeren har spart gjennom appen og utregning av hvor mye summen tilsvarer i andre kjøp, for eksempel antall kaffekopper, ble lagt til i sprint 2. Sluttproduktet oppfyller alle akseptansekriteriene til denne brukerhistorien.

Som bruker ønsker jeg å motta persontilpassede utfordringer slik at det blir enklere å spare.

I sprint 1 ble det laget en algoritme for å generere utfordringer til den enkelte bruker basert på kontoutskriftene deres og forhåndsdefinerte kategorier. Brukeren kan velge å godta eller avslå utfordringen. Algoritmen ble videreutviklet i sprint 2 slik at utfordringene ble mer realistiske og tok villighetsnivået til å endre vaner med i betraktningen. Ved høyere villighet til vaneendring, vil en bruker få utfordringer som krever mer. En bruker med lav villighet kan f.eks bli utfordret til å redusere antall innkjøp av klær fra 5 til 4 per måned, mens et høyere villighetsnivå vil kunne bli utfordret til å redusere til 2 innkjøp. I tillegg ble algoritmen utvidet til å gi et bredere spekter av utfordringer, basert på de forskjellige transaksjonene brukeren la inn på sin brukerkonto via opplasting av csv-filen. I sluttproduktet kan en bruker få persontilpassede utfordringer, som er generert ut fra transaksjonene som lastes opp via csv-filen. Dermed er dette akseptansekriteriet oppfylt i sluttproduktet. Algoritmen har likevel forbedringspotensial og er en av tingene som hadde vært naturlig å jobbe videre med. Blant annet vil algoritmen bare gi deg utfordring innenfor samme utgiftskategori selv om du avslår utfordringen og generer en ny.

Som bruker ønsker jeg å kunne sette opp et enkelt budsjett for å planlegge og disponere pengene best mulig og dermed kunne spare mer.

Det ble ikke laget funksjonalitet for at en bruker skulle kunne sette opp et budsjett. I stedet ble en oversikt over utgiftene og inntektene til brukeren implementert. Utgiftene til brukerne ble kategorisert i blant annet kategoriene 'dagligvarer', 'underholdning' og 'reise'. Dette gir brukeren en enkel, men informativ oversikt over forbruk og hvor en eventuelt har rom for å spare. Å bestemme om en transaksjon var inntekt eller utgift ble gjort under prosesseringen av den opplastede csv-fila, ved å sjekke hvilken kolonne tallet kom fra. Siden de forhåndsprogrammerte bankene hadde et gitt format på kontoutskriften, kunne det enkelt settes opp om transaksjonen gikk inn eller ut av konto, og i tillegg tilegne den en kategori basert på transaksjons beskrivelsen fra kontoutskrifta. Dette var ferdig i løpet av sprint 1, og i sprint 2 ble det lagt til funksjonalitet for å kunne velge hvilket tidsrom man vil se utgiftene i. Selv om funksjon for å sette opp og planlegge et budsjett ikke er med i sluttproduktet, ble det implementert en enklere erstatning som heller gir brukeren en rask oversikt over deres forbruksvaner. Dette var et punkt som ble diskutert med produkteier, og resultatet var enighet om at et budsjett i grunnen var utenfor det de anså som nødvendig og ønskelig. PO sitt fokus var mer på å få en bruker i gang med sparing og forhåpentligvis gjøre det til noe interessant og morsomt, heller enn å drive aktiv planlegging av personlig økonomi. De var enige i at en enkel, men informativ oversikt ville være et fornuftig alternativ, spesielt gitt målgruppen til produktet. Dermed anses dette akseptansekriteriet som oppfylt i sluttproduktet.

I sprint 2 ble også følgende brukerhistorier tatt med i produktkøen:

Som bruker ønsker jeg å få badges for utførte mål for å få motivasjon og en følelse av oppnåelse.

Etter å ha mottatt tilbakemeldinger fra brukertestene av wireframen, ble det tydelig at plasseringen av badges på profilsiden ikke var optimal. Basert på denne tilbakemeldingen ble det derfor besluttet å opprette en egen side dedikert til badges i stedet. I Sprint 2 ble deretter kriteriene som må oppfylles for å motta en badge definert. På den nye badges-siden vises alle badgesene i form av bilder med tilhørende beskrivelser. Badgene som brukeren har

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

oppnådd, vises i farger, mens de resterende vises i gråtoner, slik at det er enkelt å skille mellom dem. Informasjon om fremgangen mot å oppnå de ulike badgesene blir lagret i databasen og dersom en bruker har oppnådd et badge blir det opprettet en kobling mellom badge og bruker i databasen. Alle akseptansekriteriene til denne brukerhistorien er oppfylt i sluttproduktet.

Som bruker ønsker jeg å visuelt se status på utfordringene mine for å se fremgang.

Informasjon om utfordringene og brukerens progresjon var allerede implementert etter sprint 1, så denne brukerhistorien handlet om det visuelle aspektet av Sparestien. Sparesti-siden viser sammenhengen mellom alle målene og utfordringene en bruker har og har hatt. Fremgangen i utfordringene vises ved å fylle opp en prosentandel av sirkelen som representerer hver utfordring etter hvert som progresjonen øker. Et av akseptansekriteriene var at brukeren må registrere kjøpene sine for å spore fremgangen på utfordringene sine. Dette ble ikke implementert, men endret til at progresjonen på utfordringene oppdateres automatisk ettersom brukeren laster opp flere kontoutskrifter.

Som bruker ønsker jeg å kunne få mer kunnskap rundt personlig økonomi for å være mer bevisst på eget forbruk og bruke dette til å spare mer.

Brugerhistorien var ikke satt opp i produktkøen til sprint 1, men visning av enkle tips til på Hjem-skjermen ble implementert i sprint 1. Disse tipsene var fra det helt enkle, til det litt større. Fra å bevisstgjøre en bruker om enkle grep en kunne ta for å spare litt her og der, som for eksempel å ta med matpakke og vannflaske på skolen i stedet for å kjøpe mat i kantina hver dag, til tips om at jevnlig og langsiktig sparing i fond kan være lurt å vurdere. Disse tipsene ble valgt ut basert på kunnskapsnivået brukeren satte i profilen sin, slik at de skulle oppleves mer relevant for de forskjellige brukertypene. I sprint 2 ble økonominyheter hentet fra E24 vist på Hjem-siden. Dette var funksjonalitet som PO var spesielt interessert i. Videreutvikling av applikasjonen ville vært å kunne hente inn nyheter fra flere forskjellige kilder, og tilpasset dem til den enkelte brukers kunnskapsnivå på samme måte som tipsene. Siden både tips og nyheter ble implementert på en tydelig måte, anses dette akseptansekriteriet som oppfylt i sluttproduktet.

Etter tilbakemeldinger fra brukertester på mvp-en fra sprint 1, se vedlegg 11 og 12, ble teksten på knappene endret til fet skrift, litt informasjon om hva Sparesti er ble lagt til på forsiden, og informasjon om at man kan fjerne kategorier fra oversikten over utgiftene ble lagt til ved smultring-diagrammet. I tillegg ble navigasjonslinja flyttet til toppen av skjermen for stor skjerm, men beholdt på bunnen for liten skjerm. Noe av utseendet på profilsiden ble endret slik at det var mer intuitivt om man kunne redigere informasjonen eller ikke.

Sluttproduktet viser en rekke funksjonaliteter som støtter brukernes sparemål og økonomiske planlegging på en brukervennlig måte. Produktet tillater brukere å opprette sparemål, generere personlige utfordringer relatert til disse målene, og gi en detaljert oversikt over både samlet og individuell spareprogresjon gjennom sparestien. Muligheten til å motta badges og tilgang til kunnskap om økonomi gjennom tips og økonominyheter gir ytterligere motivasjon og innsikt. Tilknytning til nettbank viste seg å ikke være gjennomførbart, og derfor ble tilknytning til bankkonto eller innlogging med BankId ikke tatt med i løsningen. Brukere registreres og logger inn med e-postadresse og passord. Andre innloggingsmuligheter ble ikke tatt med, men det er mulig å få tilsendt nytt passord på e-post hvis man har glemt passordet. Konfigurasjonsmulighetene ved registrering, inkludert valg av kunnskapsnivå og villighetsnivå for vaneendringer, legger til rette for en tilpasset opplevelse. Muligheten for å opprette et budsjett ble endret til å vise en oversikt over brukerens utgifter. Til tross for at visse elementer fra visjonsdokumentet ikke ble inkludert, som brukerens ønsker for spareutfordringer og muligheten for felles sparemål, oppfyller produktet likevel kjernefunksjonene for personlig økonomistyring på en intuitiv måte, og funksjonene produkteierne prioriterte høyest er tatt med.

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

4.2 Administrative resultater

En viktig del av denne oppgaven var å lære hvordan man planlegger sprinter og bruker scrum aktivt under arbeidet. De første dagene ble derfor satt av til planlegging, noe som var viktig for å sikre mer målrettet og effektivt arbeid. I løpet av disse dagene ble teamet enig med produkteierne om hvilke user stories som skulle inkluderes i produktets backlog, se vedlegg 4. Dette dannet grunnlaget for oppgavene i sprintkøene for de to sprintene, se vedlegg 5 og 6.

Før sprint 1 retrospektiv tok teamet en tilstandssjekk. Dette går ut på å sjekke hvor komfortable alle er med å dele meninger og snakke høyt i teamet. Testen kjøres ved at alle anonymt skriver ned et tall fra 0 til 4 og så regnes gjennomsnittet ut. Resultatet til teamet ble 3,125, se vedlegg 16. Dette resultatet tilsier at teamet i snitt er komfortable med å dele det meste med teamet, men ikke alt.

Etter begge sprintene ble det gjennomført retrospektiver hvor teamet reflekterte over erfaringer fra sprinten. Resultatene fra sprint 1 viste at teamet var fornøyd med samarbeidet og kommunikasjonen, spesielt med hensyn på kunnskapsdeling og lære fra hverandre på en positiv og ikke-belærende måte. Det kom også fram at teamet hadde blitt mer trygge, samtidig som alle forble målrettede og arbeidsvillige. Noe som stod i kontrast til de første dagene da de fleste var litt tilbakeholdne og forsiktige. Fokusområdene som teamet kom fram til var oftere stand-ups sånn at alle hadde mer oversikt og visste hva som måtte gjøres for å komme i mål. Noe hele teamet var enige om var at oppgavene i Sprint 1 ble for store og krevde for mye arbeid å fullføre. Konsekvensene av dette ble at man brukte issue bordet lite aktivt og at man fikk en følelse av å aldri bli ferdig med det man holder på med. Teamet var også enig om at det burde ha blitt laget en burndown for å se fremgang underveis i sprinten og at det var dumt at man bare lærte enten frontend eller backend. (vedlegg 14)

Resultatene fra sprint 2 retrospektivet viste at teamet hadde tatt til seg de tidligere nevnte fokusområdene fra sprint 1. Mens det var mange av de samme punktene som ble nevnt som positive ting å fortsette med, var teamet også fornøyd med at det ble laget flere mindre oppgaver og at kommunikasjonen forbedret seg gjennom flere standups. Videre var teamet fornøyd med estimering av innsats og bruken av burndown (vedlegg 13). Teamet fant at fokusområder ville vært raskere planlegging og bedre git-struktur i frontend for eventuelle fremtidige prosjekter eller sprinter. (vedlegg 15)

5. Konklusjon og videre arbeid

5.1 Diskusjon

De første dagene av prosjektet ble satt av til planlegging. Dette var første gang hele teamet var samlet, og så oppgaven for første gang. Usikkerhet rundt oppgaven og planleggingsprosessen, i tillegg til å være samlet med mange nye mennesker, førte til en stillhet i rommet. I begge sprint-retrospektivene, se vedlegg 14 og 15, ble det påpekt at planleggingsfasen kunne kommet litt raskere i gang, men de viser også at teamet var fornøyd med kommunikasjonen og samarbeidet underveis i arbeidsperiodene. Før selve prosjektarbeidet begynte ble det holdt et kontraktsmøte hvor forventningene til hverandre og prosjektet ble diskutert, og utarbeidet en kontrakt internt i gruppen, se vedlegg 17.

Under planleggingen av sprint 1 ble det estimert hvor omfattende hver oppgave var ved å sette et antall timer per oppgave. Denne metoden viste seg å ikke fungere så godt da det skulle settes opp en burndown-graf, og derfor finnes det ikke en burndown for sprint 1. Dette ble tatt lærdom av til sprint 2, hvor estimering av arbeidsmengde for hver oppgave ble gjort med tall fra en til ti i stedet. Det ble brukt en poker-metode for å estimere, hvor hver person rekker opp hånda og viser et antall fingre som representerer tall fra 1 til 10, slik at alle kunne tenke selv før de lot seg eventuelt påvirke av andre. Det ble også satt referansepunkt ved at teamet sammen estimerte poeng til allerede fullførte oppgaver fra Sprint 1 for å kalibrere poenggivningen. Om en oppgave ble estimert til mer enn 5, ville denne bli delt opp i mindre deloppgaver. Før prosjektet ble iverksatt, ble det også laget et estimat på arbeidsinnsatsen til hver person, se vedlegg 19.

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Visjonsdokumentet (Sparesti, 2024), beskriver brukerens behov og funksjonelle egenskaper som prosjektet skulle bygges på. Produkteierne kunne deretter velge hva de anså som viktigst og hva som skulle prioriteres ned. Dette førte til at produkteierne var med på planleggingsfasen av prosjektet og hver enkelt sprint. Produkteierne hadde ikke mange eller konkrete meninger, og ønsket primært å holde seg nært visjonsdokumentet. Teamet fortalte at det var behov for prioriteringer av oppgaver på grunn av tidsbegrensninger. Det ble enighet om å prioritere funksjonaliteter som hadde høy prioritet i visjonsdokumentet, inkludert opprettelse av sparemål, mottak av personlige tilpassede utfordringer, oversikt over spart beløp og et enkelt brukergrensesnitt. Både teamet og produkteierne begrunnet dette valget med at disse funksjonene var avgjørende for å oppfylle grunntanken i prosjektets visjon. For eksempel ville applikasjonen "Sparesti" ikke vært fullstendig uten funksjoner som sparemål og oversikt over spart beløp. Produkteierne understreket også viktigheten av et intuitivt og brukervennlig grensesnitt som kunne passe brukere i alderen 18-60, uavhengig av teknologisk bakgrunn. I tillegg ønsket de å prioritere funksjoner som nyheter og tips for å øke kunnskapen og interessen for sparing, samt bruk av badges for å motivere brukeren. De viste til andre apper som brukte badges, hovedsakelig språk-appen Duolingo, for måloppnåelse som inspirasjon og mente at dette kunne ha en positiv innvirkning på sparingen. Etter diskusjoner i Sprint 1, ble ønsket om å kunne sette opp et enkelt budsjett endret til en oversikt over brukte penger, da dette ble ansett som viktig både visuelt og tekstlig. Liste over prioriteringer ligger i backlog (Se vedlegg 4)

På grunn av tidspress måtte noe av funksjonaliteten prioriteres bort. Under dette prosjektet ble samhandling med andre brukere valgt bort. Dette inkluderte funksjoner som å spare mot felles mål og dele spareutfordring med andre brukere, som var beskrevet i visjonsdokumentet. Beslutningen ble basert på produkteiernes ønske om å sikre at programmet var stabilt og kunne dekke kjernebehovene til spareapplikasjonen, heller enn å fokusere på muligheten for å dele progresjon og mål med andre.

Et av de viktigste punktene hvor produktet kommer til kort, med hensyn på visjonsdokumentet, er manglende integrasjon med nettbank. En stor del av den forespeilede applikasjonen tok utgangspunkt i at banktransaksjoner som brukeren gjorde ville kunne dukke opp i appen i sanntid. Dette slik at sparemål og utfordringer ville kunne oppdateres automatisk basert på f.eks varekjøp, at brukeren kunne gjøre direkte innskudd til en sparekonto når de jobbet på utfordringer og at alt dette kunne vises i grensesnittet for å både informere og motivere. Det fremkom av visjonsdokumentet at det var en antagelse fra produkteiers side om at innlogging med BankID var en slags portal inn i nettbanken, hvilket ikke stemmer med virkeligheten. Denne typen funksjonalitet lå altså langt utenfor det som var realistisk og praktisk mulig å få til i dette prosjektet. Det var en mulighet å benytte testbrukere fra DNB sitt utvikler-API, men disse ga ikke mulighet til å legge inn transaksjoner på en slik måte som krevdes for å teste og demonstrere funksjonaliteten i produktet. Dette ble tatt opp med produkteier, hvor det ble bestemt at manuell opplasting av kontoutskrifter, som ble lest inn i en database og koblet til brukeren, ville fungere som en god erstatning, selv om dette ville medføre at enten måtte brukeren bruke appen mer aktivt til å loggføre pengebruken, eller at det måtte lastes opp nye kontoutskrifter med jevne mellomrom. Dermed ble databasen satt opp til å håndtere flere bankkonti, med tilhørende transaksjoner, på en bruker, men kontofunksjonene ble følgelig ikke utnyttet i sin helhet. Resultatet var at produktet ikke fungerer like friksjonsfritt som ville vært ønskelig, og er avhengig av mer brukerinteraksjon.

I nåværende versjon av programmet oppdateres utfordringer automatisk når nye kontoutskrifter blir lastet opp av brukeren. Algoritmen går gjennom alle transaksjoner, hvis en transaksjon matcher kategorien og tidsintervallet til en utfordring vil utfordringen bli oppdatert for å ta med de nye transaksjonene. Når brukeren navigerer til sparesti-siden vil målobjektene sjekke om utfordringene har endret seg. Siden teamet ikke har fått tilgang til bank API så må transaksjonene lastes opp manuelt som csv-filer av brukeren, med integrering av bank API vil dette skje automatisk og brukeren vil kun trenge å godta at Sparesti henter transaksjoner for brukeren.

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

Produkteierne hadde opprinnelig sett for seg at progresjonen på utfordringer skulle registreres manuelt, noe som også kommer frem i visjonsdokumentet (Sparesti, 2024). Hvis en utfordring for eksempel var å redusere antall kaffekjøp på en uke fra 5 til 3, ville de at man skulle registrere de dagene man ikke hadde kjøpt kaffe. Denne løsningen ble diskutert, men det ble bestemt at det innebar for mye brukerinteraksjon. Hvis en bruker hadde veldig mange utfordringer pågående samtidig og alle skulle kreve registrering av handlinger daglig, ville det blitt plagsomt og tungvint å bruke appen. Nå skjer oppdatering av progresjonen automatisk ved opplasting av nye kontoutskrifter, og brukeropplevelsen er derfor mer sømløs.

Som følge av at innlogging med Bank-ID ikke gav tilgang til nettbanken, valgte gruppen og produkteierne å nedprioritere BankID som innloggingsmetode, og heller fokusere på kjernefunksjonalitet. Det ble brukt noe tid på å implementere innlogging på andre måter med for eksempel Google eller Facebook gjennom OAuth2, men også det ble nedprioritert for å kunne bruke mer tid på viktigere funksjonalitet.

Under et så stort prosjekt som dette er det mange hensyn og valg som skal tas. En viktig faktor var å følge på Web Content Accessibility Guidelines (WCAG). I visjonsdokumentet (Sparesti, 2024) er det kun retningslinje nr 2.2, prinsipp 1 nevnt, som omhandler god brukskvalitet. Dette inkluderer krav om at applikasjonen skal være intuitiv, enkel å bruke, og tilpasset målgruppen. Et av tiltakene for å sikre at disse kravene ble etterlevd var å gjennomføre mange brukertester med personer i ulike aldre og med varierende teknologisk bakgrunn.

Fokuset på farger har også vært viktig. Nettsiden WebAIM.org ble brukt for å sjekke kontrasten mellom ulike fargekombinasjoner. I tillegg ble det valgt farger og symboler som er universelt anerkjent, for eksempel bruk av rødt for avslag og grønn for godkjenning. Rød og grønn kan være vanskelig for fargeblinde å se forskjell på, men alle knapper har tekst slik at det skal være mulig å skille dem fra hverandre og forstå funksjonen. Det ble også lagt vekt på å gjøre det visuelt tydelig når elementer er interaktive, for eksempel ved å endre musepekeren fra en pil til en pekende hånd og ved å markere interaktive elementer med endrede farger eller understreket tekst. Størrelsen på tekst har vært nøye vurdert for å sikre god lesbarhet for alle brukere. Teksten er stort sett satt til 16 piksler, hvilket er en anerkjent standard størrelse for slike brukergrensesnitt. Overskrifter og annen tekst på applikasjonen er formulert kort og presist, samtidig som de er forståelige for alle. Dette gjelder også feilmeldinger, hvor feilmeldinger er forsøkt skrevet både konsist og beskrivende, samt klare instruksjoner for å rette opp feilene. Innskrivingsfeltet viser tydelig hva som skal skrives inn, og hvilket format inputen forventes å ha.

Etter en diskusjon med fagveileder ble det besluttet at alle frontend-testene skulle utføres som end-to-end (E2E) tester i Cypress. Det ble valgt å ikke inkludere enhetstester, da E2E-tester i seg selv gir tilstrekkelig dekning for å sjekke om elementer og komponenter er til stede på siden som forventet. Ved å for eksempel simulere at en knapp blir trykket på i E2E-testene, kan det konstateres at knappen eksisterer på siden. Siden det ikke ble skrevet enhetstester, er det vanskelig å tallfeste testdekningen. Likevel anses testdekningen som god, og dekker de viktigste funksjonalitetene. Uten nøyaktige tall er det vanskelig å si noe konkret om prosentvis dekning, men gitt testene som ble skrevet, estimeres det at testdekning overskrider 30%, i samsvar med kravene i oppgaven.

En annen ting verdt å nevne er at pipeline kun ble implementert for backend-delen av applikasjonen og ikke for frontend-delen. Dette fordi fokuset i utviklingsprosessen naturlig nok lå på de mer konkrete og kritiske komponentene, og dermed ble behovet for pipeline i frontend oversett. Selv om dette representerer en klar mangel i prosjektet som helhet, har det likevel en positiv læringseffekt ved at det skjerper oppmerksomheten til fremtidige prosjekter, med hensyn på å inkludere slike elementer som har en mer støttende og underliggende funksjoner. Implementeringen av en pipeline for frontend ville ha vært en forbedring av utviklingsprosessen ved å automatisere bygge- og testprosesser, øke kvaliteten på koden gjennom kontinuerlig integrasjon, og muliggjøre raskere iterasjoner og levering av frontend-funksjoner til sluttbrukerne.

5.2 Videre arbeid

Ved videre utvikling av produktet, hadde det vært naturlig å søke om tilgang til nettbanker, slik at systemet kunne fungere mer i tråd med visjonen. Med dette på plass, er mulighetene for å utvide både mål, utfordringer og merker mye større og lettere å jobbe mot. Etter nettbank, ville det mest nyttige vært å få tilgang til API fra

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

kundeprogrammer, blant annet Coop og Rema 1000 “Æ”. Disse lagrer digitale kvitteringer for kjøp som kunden utfører, hvilket ville gitt detaljert informasjon om kjøpshistorikk. Det ville da vært mye enklere å utvikle en tjeneste som var mye mer nøyaktig i de utfordringene en bruker fikk, siden det ville vært mulig å identifisere enkeltvarer som snus eller sjokolade, som ville vært lavhengende frukt hva sparing angår. En annen lignende mulighet ville vært å inngå avtaler med både butikker og leverandører av kassasystemer, som Lindbak og Visma/Extenda Retail. Med detaljerte og automatiske oppdateringer av en brukers transaksjoner, ville mye av hovedfunksjonaliteten vært på plass. Videre ville finpussing av algoritmen som genererer utfordringer vært på sin plass, siden tilgang til nettbank og kvitteringer ville gitt store muligheter for enda bedre personlig tilpasning. Med dette på plass ville finpussing av det visuelle rundt Sparestien og utfordringene vært viktig for å videreutvikle "gamification" av sparingen, og gi brukeren enda bedre oversikt over fremgang og bedre visuell respons på denne.

Samtidig med denne utviklingen, ville det være nyttig å tilpasse nyheter basert på brukerens kunnskapsnivå. Dette ville gjøre nyhetene mer relevante og interessante for brukeren. For eksempel kunne brukere med høyt kunnskapsnivå få mer avanserte nyheter, mens de med lavere kunnskapsnivå kunne få mer forståelige og mindre tekniske nyheter.

En annen nyttig funksjon å legge til ville være muligheten til å dele prestasjoner og mål med andre brukere. For eksempel kunne brukere dele hvilke merker (badges) de har oppnådd med andre brukere av appen, eller til og med dele sine prestasjoner på sosiale medier som Facebook. Dette åpner for samarbeid, spesielt for familier eller venner som sparer sammen til felles mål, som for eksempel en ferie.

Det ville også vært mulig å implementere et leaderboard, hvor brukerne kunne bli rangert basert på hvor mye penger de har spart ved å bruke appen. Dette kan føre til mer engasjement og konkurranseinstinkt blant brukerne, og dermed inspirere dem til å spare enda mer.

5.3 Konklusjon

Prosjektet resulterte i en funksjonell spare-applikasjon som produkteier har sagt seg fornøyd med, selv om noen funksjoner ikke ble implementert. Hovedmålene som ble oppnådd inkluderer:

- Opprette sparemål.
- Motta tilpassede utfordringer.
- Få oversikt over inntekter, utgifter og spart beløp.
- Motivasjon og kunnskap gjennom badges, integrerte nyheter og personaliserte tips.
- Et brukervennlig grensesnitt.

Integrasjon med nettbank og deling av spareutfordringer med andre brukere ble derimot ikke gjennomført på grunn av teknologiske og tidsmessige begrensninger.

Gjennom prosjektet har teamet lært mye om viktigheten av god planlegging og kommunikasjon. Til tross for en treg start i planleggingsfasen og usikkerheter rundt produkteiers visjon vedrørende BankID og bank API, har teamet forbedret seg gjennom retrospektiver og tilpasning av arbeidsmetoder, som bruk av poker-metoden for oppgaveestimering og statusoppdateringer gjennom dagen. Dette har vært med på å styrke samarbeidet og effektiviteten i teamet.

Prosjektets suksess kan tilskrives flere faktorer som god kommunikasjon i teamet gjennom møter og åpenhet rundt problemer, gode rutiner som følge av Scrum metodikken, tilbakemelding fra produkteierne og andre tredjeparter gjennom brukertesting og god innsats og initiativ fra teammedlemmene.

Selv om noen funksjoner ikke ble implementert, har applikasjonen nå et solid fundament som kan videreutvikles.

Samlet sett har prosjektet ikke bare resultert i en funksjonell applikasjon, men også gitt teamet verdifulle erfaringer og innsikt i smidig utvikling, teamarbeid, og innsikt i hvordan det er å arbeide i et mer karriererettet prosjekt. Dette er erfaringer som teammedlemmene kan dra nytte av i fremtidige prosjekter og karrierer.

Team 8	Versjon: 3.0
Sluttrapport	Dato: 15/05/24

6. Referanser

- Agile Alliance (2001). Manifestet for smidig programvareutvikling. <https://agilemanifesto.org/iso/no/manifesto.html>, besøkt 12.05.2024.
- Aune, Billingstad, Christiansen, Eriksen, Røe (2023). *IDATT1002 Systemutvikling 1 - Prosjektoppgave "Baecon" 2023 Team 1 - Wiki-struktur og -oppsett*. https://gitlab.stud.idi.ntnu.no/team_01-idatt1002/project-assignment-idatt1002-y2023_spring-t01/-/wikis/home, besøkt 13.05.2024.
- Figma (u.å.). *What is wireframing?*. <https://www.figma.com/resource-library/what-is-wireframing/> besøkt 12.05.2024
- Fink, G., Flatow, I., & Group, S. (2014). *Pro Single Page Application Development : Using Backbone.js and ASP.NET* (1st ed. 2014.). Apress : Imprint: Apress.
- Givær, A. F. (u.å.). *En guide til brukertesting*. <https://prototypen.no/brukertesting>, besøkt 12.05.2024
- Hansen, T. H. & Mallaug, T. (2003). *Databaser*. (2. utg.) Gyldendal Akademisk og Stiftelsen TISIP
- IDATT2106 - Prosjekt (2024). *Samleforelesning* [Lysarkpresentasjon] Blackboard. <https://ntnu.blackboard.com/ultra/institution-page>
- Produkteiere (2024). *Visjonsdokument Sparesti*. https://ntnu.blackboard.com/ultra/courses/_40197_1/cl/outline
- SmartBear Software Inc. (2024). *Swagger Core* (Versjon 2.2.21) [Programvare]. SmartBear Software Inc. <https://swagger.io>.
- Tilsynet for universell utforming av ikt (u.å. a). *Informasjon om tilsynet sine oppgaver*. <https://www.uutilsynet.no/om-oss/oppgaver/279>, besøkt 12.05.2024.
- Tilsynet for universell utforming av ikt (u.å. b). *Universell utforming i utviklingsprosessen*. <https://www.uutilsynet.no/regelverk/universell-utforming-i-utviklingsprosessen/201>, besøkt 12.05.2024.
- Tilsynet for universell utforming av ikt (u.å. c). *WCAG-standard*. <https://www.uutilsynet.no/wcag-standard/wcag-standard/86>, besøkt 12.05.2024.
- Wolford, B. (u.å). *What is GDPR, the EU's new data protection law?*. GDPR.eu. <https://gdpr.eu/what-is-gdpr/> besøkt 12.05.2024.